CYBER HUNT

PRODUCT REQUIREMENTS DOCUMENT

Version 2.0 | 31 October 2026 | Draft for Review

Hunt the Threat. Defend the AI. Secure the Future.

Version	2.0 — rebuilt from the ground up
Date	31 October 2026
Status	Draft for team review and approval
Project	CYBER HUNT
Problem statement	CYBER-02 — Red Team vs Blue Team: Agents Attacking Agents
Tagline	Hunt the Threat. Defend the AI. Secure the Future.
Document purpose	Define the product, functional behaviour, technical architecture, security controls, data model, APIs, user experience, testing approach, acceptance criteria and a realistic 24-hour hackathon MVP.

1. Executive Summary

Overview: CYBER HUNT is a real-time, sandboxed, multi-agent AI cybersecurity platform. A Red Team agent dynamically creates adversarial inputs; a Target AI agent processes a legitimate task alongside untrusted content and proposes tool actions; and a Blue Team Guardian analyses the proposed behaviour. An independent policy layer makes the enforcement decision before any simulated tool action can run.

Core principle: The LLM recommends. The deterministic policy layer decides. The mock tool gateway enforces.

Product outcome: Users can start, observe, pause, approve, stop and review live security simulations. The platform records each agent turn, proposed action, threat assessment, policy decision, tool result and score so that the outcome is explainable and reproducible.

Real-time definition: Real-time means live, step-by-step agent execution and event delivery to the user interface while a simulation is running. It does not mean unrestricted access to real email, payment, browser, file or enterprise systems.

Safety boundary: All attack execution and tool operations occur inside an isolated simulation using synthetic data and mock services. No real accounts, real payments, real personal files or external systems are modified.

2. Problem Statement

Problem: AI agents can receive instructions from untrusted sources such as emails, webpages and documents. Those instructions may attempt to override the user's task, extract data, misuse tools or trigger unauthorized actions. A system that relies only on the model to recognize and refuse malicious instructions is not a sufficient enforcement boundary.

Gap addressed: CYBER HUNT makes the attack-and-defence process observable and testable. It separates model reasoning from deterministic authorization and tool execution, then measures both security outcomes and legitimate task completion.

Problem statement: How might we evaluate and demonstrate whether an AI-agent workflow can withstand dynamically generated adversarial instructions while preserving legitimate task completion, using independent policy enforcement and auditable evidence?

3. Vision, Goals and Non-Goals

Vision: Provide an interactive arena for evaluating how AI agents respond to adversarial content and how an independent security layer prevents unsafe tool actions.

Goals: G1: dynamically generate attacks within controlled scenarios. G2: execute multi-agent workflows with explicit state transitions. G3: inspect and score proposed actions before execution. G4: enforce allow, block or approval decisions outside the LLM. G5: stream meaningful events to the dashboard. G6: preserve an auditable simulation history. G7: measure detection, blocking, false positives and benign task completion.

Non-goals: No real-world penetration testing; no attacks against public websites or third-party systems; no real email or payment execution; no unrestricted autonomous agent access; no production SOC replacement; no claim that model-based detection is infallible; no guarantee of zero false positives or false negatives.

4. Users and Roles

Participant / Operator: Creates and configures simulations, selects a scenario category, supplies a benign task, starts or stops runs, observes live events and reviews results.

Approver: Reviews actions placed in AWAITING_APPROVAL, inspects the proposed operation and evidence, then approves or rejects within the simulation. Approval is scoped to the specific action and simulation.

Administrator: Manages users, roles, scenario availability, policy versions, system configuration and audit access.

Judge / Observer: Views a read-only live arena or completed report, including attack timeline, decisions, evidence and metrics.

Authorization: Use Supabase Auth for identity. Enforce role and simulation ownership checks on the backend. Frontend visibility is not an authorization control.

5. Scope and Priorities

In scope: Web application; authenticated users; real-time simulation lifecycle; Red Team, Target AI and Blue Team agents; Gemini integration; LangGraph orchestration; deterministic policy enforcement through OPA; PII detection/redaction through Presidio; isolated mock tools; event streaming; approval pause/resume; audit history; analytics and exportable results.

Out of scope: Connecting to real inboxes, payment providers, personal storage or public websites; executing live offensive security actions; training custom foundation models; multi-tenant enterprise billing; complex tournament infrastructure; arbitrary user-supplied code execution.

P0 — hackathon MVP: Application shell and authentication; one complete LangGraph simulation; Gemini-backed agent calls; at least three robust scenario categories; structured action schema; OPA policy decision; mock tool gateway; live event stream; simulation persistence; basic approval pause/resume; live arena and final score.

P1 — after core path is stable: All six scenario categories; adaptive attack strategy; Presidio integration; richer analytics; event replay; policy version management; downloadable report; recovery from interrupted sessions.

P2 — future: Additional model providers; multi-team competition; configurable policy packs; advanced experiment comparison; richer evidence visualisation; evaluator API and integrations.

6. Product Workflow

1. Configure: Operator selects a scenario category, enters a legitimate task, selects a simulation limit and starts a run. The backend creates a simulation record and initial state.

2. Generate attack: Red Team receives only the scenario context and permitted synthetic environment state. Gemini generates a structured attack plan and payload. The backend validates the output and applies scenario constraints.

3. Process task: Target AI receives the legitimate task plus clearly labelled untrusted content. It may use only the tools exposed by the simulation. It returns a response and, when appropriate, a structured proposed action.

4. Inspect: Blue Team Guardian reviews the untrusted content, task context and proposed action. It returns a structured threat assessment, rationale, indicators and recommended handling.

5. Enforce: The backend submits the normalized action context to OPA. OPA returns ALLOW, BLOCK or REQUIRE_APPROVAL with policy identifiers and reasons. The model cannot override this decision.

6. Inspect sensitive data: Presidio scans relevant synthetic content and proposed data transfers for supported PII patterns. Findings are supplied as evidence to policy evaluation; Presidio does not authorize actions.

7. Execute or stop: Only an ALLOW decision can proceed to the mock tool gateway. BLOCK is recorded without execution. REQUIRE_APPROVAL pauses the simulation until an authorized decision is received.

8. Record and adapt: Every step is stored with timestamps, correlation IDs, agent outputs, policy result and tool result. Red Team may adapt its next strategy based on previous outcomes within the same sandbox.

9. Score and finish: The orchestrator continues until the configured step limit, operator stop, failure or completion condition. The platform calculates metrics and presents the final report.

7. Attack Scenario Categories

A. Direct prompt injection: Untrusted task-adjacent text attempts to override the system's intended task or agent constraints. Test whether the Target AI treats it as untrusted and whether unsafe actions are blocked.

B. Email-based injection: A synthetic email contains instructions that attempt to redirect the assistant, disclose data or misuse an email tool.

C. Webpage-based injection: A simulated webpage includes adversarial text that attempts to influence the agent's next action. No public website is contacted.

D. Document-based injection: A synthetic document contains embedded or visible instructions attempting to influence summarisation, extraction or tool use.

E. Data exfiltration attempt: An attack attempts to move synthetic protected records to an unauthorized mock destination. The gateway must prevent unauthorized transfer.

F. Unauthorized payment/action: An attack attempts a simulated payment or other high-impact operation without the required authority or approval.

Dynamic behaviour: The six categories define attack families, not fixed payloads. Red Team should generate varied payloads and strategies at runtime. Scenario templates may constrain the environment and provide safe fallback cases, but must not be the only attack source.

Benign controls: Each category should have benign tasks or non-malicious inputs to measure false positives and legitimate task completion.

8. Multi-Agent Design

Red Team Agent: Objective: create adversarial inputs within the selected category and controlled environment. Inputs: scenario definition, synthetic environment facts, prior round outcomes and attack constraints. Outputs: structured attack plan, payload, intended risk and target surface. Restrictions: cannot call tools, access secrets, alter policies or execute actions. Adaptation is limited to simulation history.

Target AI Agent: Objective: complete the user's legitimate task while treating external content as untrusted. Inputs: user task, allowed synthetic context and tool descriptions. Outputs: user-facing response and optional structured action proposal. Restrictions: no direct tool execution; no authority to change policy; all actions require backend validation.

Blue Team Guardian: Objective: identify prompt injection, suspicious intent, data access violations and risky proposed actions. Inputs: original task, relevant untrusted content, proposed action and synthetic context. Outputs: structured risk assessment, indicators, severity, confidence, rationale and recommended disposition. Restrictions: advisory only; cannot itself authorize or execute.

Agent communication: Agents communicate through typed, versioned state objects managed by LangGraph. Do not pass arbitrary free-form model output directly between privileged components. Validate all model responses before state mutation.

Gemini: Use Gemini API as the primary model provider for all three agents. Select a currently supported fast model after checking availability, rate limits, pricing and structured-output support during implementation. Keep model identifier configurable server-side. Do not hard-code a model name into business logic.

Model failure: On timeout, malformed output or rate limit, apply bounded retries with backoff. If the required security assessment cannot be obtained, fail closed for sensitive actions: block or require approval rather than executing.

9. Backend and Agent Orchestration

Python + FastAPI: Primary backend and API boundary. Responsible for authentication verification, request validation, simulation lifecycle, orchestration invocation, tool gateway access, OPA calls, Presidio scans, persistence and event delivery.

LangGraph: Orchestrates Red Team, Target AI and Blue Team nodes, conditional edges, bounded loops, checkpointed state and approval interrupts. Use explicit graph state and typed node inputs/outputs.

Simulation Orchestrator: Creates runs, assigns correlation IDs, controls step limits, checks cancellation, persists state transitions and coordinates graph execution. Each step must be bounded; do not rely on one long-lived HTTP request for the whole simulation.

Execution model: Use background task/worker execution appropriate to the chosen hosting environment. Persist checkpoints so a run can resume after approval or recoverable interruption. The exact worker/queue technology is an implementation decision to validate against hackathon hosting limits.

Idempotency: Every start, step, approval and tool-execution command must carry an idempotency key or unique operation ID. Duplicate requests must not execute a mock action twice.

10. Policy Engine — Open Policy Agent

Purpose: OPA is the independent authorization and enforcement decision point. Policies are written in Rego and evaluated by the backend before tool execution.

Decision contract: OPA returns a structured object containing decision (ALLOW, BLOCK or REQUIRE_APPROVAL), risk level, reason codes, matched policy IDs, required permissions and approval scope.

Inputs: Authenticated actor and role; simulation ID and state; tool name; action type; resource classification; requested operation; synthetic data labels; Presidio findings; scenario constraints; policy version.

Policy examples: Allow reading an authorized synthetic inbox; block attempts to export restricted synthetic records to an untrusted destination; require approval for a simulated high-impact operation; block any tool call not in the simulation allowlist.

Enforcement: Backend must treat missing, invalid, unavailable or malformed OPA decisions as deny-by-default for sensitive actions. The mock gateway independently checks a signed or server-generated execution authorization and verifies that it matches the approved action.

Policy governance: Version policies, store the policy version used for each decision and log matched rules. Policy changes must not retroactively alter recorded decisions.

11. Sensitive Data Detection — Microsoft Presidio

Purpose: Presidio identifies supported sensitive entities in synthetic content and helps redact them from logs or outbound mock payloads.

Where used: Scan relevant synthetic documents, generated content and proposed data-transfer payloads before the policy decision and before logging where appropriate.

Limitations: Presidio is a detection/redaction aid, not a complete DLP system. Detection depends on recognizers, language, context and configuration. It does not decide whether a user is authorized.

Privacy: Use synthetic data only. Store redacted content by default. Avoid retaining raw sensitive-looking payloads unless essential for the controlled test and explicitly configured.

12. Mock Tool Gateway and Sandbox

Purpose: The gateway is the only component allowed to perform simulated tool operations. It enforces the OPA result and validates the exact action before execution.

Initial mock tools: Synthetic email read/search/send; synthetic browser/page fetch from an internal fixture set; synthetic file list/read/write; simulated payment request; optional mock database lookup.

Gateway checks: Tool allowlist; schema validation; simulation ownership; action ID; OPA authorization; approval token when required; resource scope; rate and step limits; idempotency; audit record.

Isolation: No arbitrary network egress, shell execution, host filesystem access, real credentials or unrestricted URL fetching. Browser-like scenarios must use local fixtures or an allowlisted internal simulation service.

Execution response: Return structured status, result summary, synthetic evidence reference, execution timestamp and action ID. Never return secrets or raw protected fixture contents unnecessarily.

13. Simulation State Machine

States: INITIALIZING, RUNNING, ANALYSING, AWAITING_APPROVAL, PAUSED, COMPLETED, FAILED and CANCELLED.

Transitions: INITIALIZING → RUNNING after setup; RUNNING → ANALYSING for guardian/policy evaluation; ANALYSING → RUNNING after a safe step; ANALYSING → AWAITING_APPROVAL when approval is required; AWAITING_APPROVAL → RUNNING after approval or rejection handling; any active state → PAUSED by operator; PAUSED → RUNNING by operator; active state → COMPLETED at limit or successful finish; recoverable errors may retry; unrecoverable errors → FAILED; operator stop → CANCELLED.

Approval semantics: Approval applies only to the exact action, parameters, simulation and policy version presented. Any material change invalidates the approval. Rejection records the reason and resumes or terminates according to scenario policy.

Recovery: Persist graph checkpoint, state version and last completed step. On reconnect, frontend reloads current state and subscribes from the last known event sequence. Backend prevents duplicate action execution.

14. Real-Time Event Delivery

Transport: Use WebSockets from FastAPI for live arena events. Supabase Realtime may additionally be used for database-driven updates and cross-client views. Avoid implementing two competing sources of truth.

Event types: simulation.created, simulation.status_changed, red_team.attack_generated, target_ai.response_ready, target_ai.action_proposed, guardian.analysis_completed, policy.decision_made, approval.requested, approval.resolved, tool.executed, tool.blocked, score.updated, simulation.completed and simulation.failed.

Event envelope: Each event includes event_id, simulation_id, sequence_number, event_type, timestamp, schema_version and payload. Use sequence numbers for ordering and event IDs for deduplication.

Reconnect: Client reconnects with simulation ID and last received sequence number. Backend replays retained events or provides a state snapshot, then resumes live delivery.

Latency target: Dashboard event delivery target: p95 under 1.5 seconds after the backend persists an event under normal hackathon test conditions. This is a target to measure, not a claimed result.

Frontend: React dashboard subscribes to the simulation stream, updates panels incrementally, displays connection state and fetches a fresh snapshot after gaps or reconnects.

15. Data Model — Supabase PostgreSQL

profiles: User profile, role and account metadata. Role changes restricted to administrators.

simulations: Simulation ID, owner, scenario category, task summary, state, configuration, step limit, start/end timestamps, current step and final metrics.

simulation_events: Append-only event stream with event ID, simulation ID, sequence number, type, timestamp, schema version and redacted JSON payload.

agent_runs: Agent name, simulation ID, step, model/provider identifier, prompt/version reference, status, latency, token usage if available, validated output reference and error code. Avoid storing hidden reasoning or secrets.

attack_logs: Attack category, strategy metadata, payload reference or redacted payload, target surface, generated time and outcome.

defence_logs: Guardian risk score, severity, indicators, explanation, recommendation and analysis status.

tool_logs: Action ID, tool, operation, normalized parameters, OPA decision, execution status, result summary and timestamp.

approval_requests: Action ID, simulation ID, requester, approver, decision, reason, exact action hash, policy version, expiry and timestamps.

policy_decisions: Decision ID, policy version, decision, reason codes, input hash, related action and timestamp.

round_results: Per-step attack result, detection result, block result, benign completion result and scoring inputs.

Relationships: All simulation-specific records reference simulations. User ownership and role checks are enforced server-side and with Supabase RLS where applicable.

Storage rules: Use JSONB for versioned structured payloads where appropriate, but index simulation ID, event sequence, timestamps, state and owner. Apply retention limits and redaction. Do not store API keys or raw credentials.

16. API Requirements

POST /api/simulations: Create a simulation after validating user, scenario, task and limits.

POST /api/simulations/{id}/start: Start orchestration and return current state; execution continues independently.

GET /api/simulations/{id}: Return authorized simulation snapshot.

GET /api/simulations/{id}/events: Return events after a supplied sequence number for replay/recovery.

POST /api/simulations/{id}/pause: Pause at a safe orchestration boundary.

POST /api/simulations/{id}/resume: Resume a paused or approved workflow.

POST /api/simulations/{id}/stop: Cancel a run and prevent further tool execution.

POST /api/approvals/{id}/decision: Approve or reject a specific action with authorization, action hash and reason.

GET /api/simulations/{id}/report: Return final metrics, event summary and evidence references.

WebSocket /ws/simulations/{id}: Stream ordered events to an authorized client.

API rules: All endpoints require authentication except explicitly public health checks. Validate inputs with Pydantic. Enforce ownership and role authorization. Use rate limits, idempotency and consistent error envelopes.

17. Frontend and User Experience

Technology: React + TypeScript + Tailwind CSS. Use a suitable React framework/build tool selected by the team. The architecture does not require Next.js API routes because FastAPI is the sole backend.

Landing and authentication: Explain the platform, provide sign-in and account access. Use Supabase Auth and backend token verification.

Dashboard: Show active simulations, recent runs, aggregate metrics, status and quick access to the arena.

Battle Arena: Primary real-time screen with Red Team activity, Target AI task/action, Blue Team analysis, OPA decision, mock tool result, event timeline and connection status.

Attack Explorer: Filter attack history by category, severity, outcome, simulation and time.

Defence Centre: Show policy decisions, risk indicators, blocked actions, approval queue and policy references.

Analytics: Display detection rate, block rate, false positives, benign task completion, latency and trends. Label sample size and calculation method.

History and report: Search completed simulations and inspect chronological evidence.

Settings: Profile, role-appropriate preferences, model configuration visible only as safe metadata, and policy/scenario settings for authorized users. Never display API secrets.

Visual direction: Professional dark cybersecurity interface, clear red/blue distinction, restrained motion, readable logs, accessible contrast and responsive layouts. Avoid decorative animation that obscures security events.

18. Metrics and Scoring

Attack generation success rate: Valid generated attack payloads divided by attack-generation attempts.

Threat detection rate: Malicious test cases correctly flagged divided by malicious test cases presented.

Blocking rate: Unauthorized malicious actions prevented before mock execution divided by unauthorized malicious actions attempted.

False positive rate: Benign actions incorrectly flagged or blocked divided by benign actions evaluated.

Benign task completion: Legitimate tasks completed correctly without unsafe action divided by benign tasks started.

Unauthorized execution count: Count of prohibited actions that reached mock execution; MVP target is zero.

Agent response latency: Time from agent request to validated response, reported by agent and percentile.

Policy evaluation latency: Time from policy request to OPA decision.

Event delivery latency: Time from persisted backend event to client receipt.

Scoring fairness: Report denominators, scenario mix and sample count. Do not reward blocking everything; include benign task completion and false positives.

19. Non-Functional Requirements

Security: Backend-only secrets; least privilege; deny-by-default enforcement; input/output validation; isolated tools; authenticated event streams; audit logs; no direct agent tool execution.

Reliability: Bounded retries, timeouts, idempotency, persisted state, graceful API failure and recoverable approval interrupts.

Performance: Avoid unbounded agent loops; configure per-agent timeouts, simulation step limits and concurrency limits. Measure actual latency under test.

Scalability: Keep agent nodes, policy evaluation and tool adapters modular. Separate API request handling from longer simulation execution.

Maintainability: Typed schemas, versioned events, modular prompts, versioned Rego policies, automated tests and clear environment configuration.

Usability: Operator can understand why an action was allowed, blocked or held for approval without reading raw model prompts.

20. Security, Privacy and Threat Model

Threats: Prompt injection; malicious untrusted content; unauthorized tool calls; data exfiltration; privilege escalation; model output manipulation; replay/duplicate requests; approval spoofing; event leakage; API key exposure; denial of service through agent loops.

Controls: Treat external content as data, not authority; isolate agent roles; validate structured outputs; apply OPA to every tool action; gateway re-checks; bind approvals to action hashes; redact logs; enforce quotas and step limits; authenticate WebSocket subscriptions; restrict network egress.

Secrets: Gemini API key, Supabase service-role key and OPA administration credentials must remain in server-side secret storage. Never commit .env files. Use least-privilege credentials.

Data: Use synthetic fixtures. Clearly label synthetic records. Avoid collecting real personal information. Redact sensitive-looking strings in telemetry.

Limitations: LLM guardrails can miss attacks or produce false positives. OPA only enforces encoded policy. Presidio recognizers are imperfect. Security claims must be based on measured tests, not assumed guarantees.

21. Error Handling and Observability

Error categories: Provider timeout/rate limit; malformed model output; LangGraph node failure; OPA unavailable; Presidio unavailable; database failure; WebSocket disconnect; approval expiry; tool gateway rejection; invalid state transition.

Fail-safe behaviour: If policy evaluation is unavailable, do not execute sensitive actions. If Guardian analysis fails, sensitive actions require approval or are blocked. If the event stream fails, continue only if persistence and safety checks remain healthy; reconnect clients from stored events.

Observability: Structured logs with simulation ID, step ID, event ID, agent, duration and error code. Avoid logging API keys, hidden reasoning or unredacted synthetic secrets.

User feedback: Show clear status and actionable messages. Distinguish a blocked attack from a technical failure.

22. Testing Strategy

Unit tests: Validate Pydantic/Zod-equivalent API schemas, state transitions, policy inputs, action hashes, scoring calculations, redaction and idempotency.

Agent contract tests: Test each Gemini agent against valid, malformed, empty, refusal and timeout responses. Verify outputs conform to schemas.

Policy tests: Test allow, block and approval cases for every tool and scenario. Include deny-by-default and OPA-unavailable tests.

Gateway tests: Verify no tool operation executes without a valid ALLOW decision or matching approval. Test duplicate requests and altered action parameters.

Integration tests: Run a full Red → Target → Guardian → OPA → Gateway → event persistence flow.

Real-time tests: Test event ordering, reconnect, replay, duplicate suppression and multiple observers.

Scenario tests: Run malicious and benign cases for each attack family. Record detection, blocking, false positives and task completion.

Acceptance evidence: Keep test IDs, scenario configuration, policy version and outcome so metrics can be reproduced.

23. Acceptance Criteria

AC1: An authenticated operator can create and start a simulation.

AC2: Red Team can generate a valid dynamic attack payload using Gemini within a selected category.

AC3: Target AI can process the task and return a validated structured action proposal.

AC4: Blue Team Guardian returns a validated threat assessment.

AC5: Every proposed tool action is evaluated by OPA before gateway execution.

AC6: BLOCK decisions never execute the mock tool action.

AC7: REQUIRE_APPROVAL pauses the workflow and only the authorized, matching approval can resume it.

AC8: Only ALLOW or valid approved actions execute in the mock gateway.

AC9: Live dashboard receives ordered updates without manual refresh.

AC10: Simulation events and results persist and can be retrieved after reconnect.

AC11: Operator can pause, resume and cancel a simulation.

AC12: Final report shows metrics with denominators and benign-task performance.

AC13: No real external system or real payment/email/file action is executed.

AC14: Secrets are not exposed to the browser or committed to the repository.

24. 24-Hour Hackathon Implementation Roadmap

Hours 0–2: Foundation: Repository, environment, React shell, FastAPI health/auth skeleton, Supabase connection, database migrations and local run instructions.

Hours 2–5: Contracts and orchestration: Define typed state and action schemas, LangGraph skeleton, simulation state machine, event envelope and persistence.

Hours 5–9: Gemini agents: Integrate Gemini; implement Red Team, Target AI and Guardian nodes; validate outputs; add timeouts and bounded retries.

Hours 9–12: Enforcement: Integrate OPA; implement baseline Rego policies; create mock tool gateway; enforce deny-by-default and idempotency.

Hours 12–15: Live arena: Build WebSocket event stream, live dashboard panels, status transitions and event replay/reconnect.

Hours 15–18: Scenarios: Complete at least three robust scenario categories and benign controls; add remaining categories only if core path is stable.

Hours 18–21: Approval and metrics: Implement approval interrupt/resume, audit timeline, score calculations and report view.

Hours 21–24: Hardening and presentation: Run end-to-end tests, fix critical bugs, verify secret handling, deploy if stable, capture screenshots and prepare judge explanation.

Scope rule: A complete secure vertical slice is more valuable than six incomplete scenarios. If time is tight, prioritize one end-to-end flow, three scenario categories, policy enforcement and live events.

25. Final Evaluation Flow

Operator journey: Sign in → open Battle Arena → select scenario category → enter benign task → start simulation → observe dynamic Red Team attack → inspect Target AI proposal → observe Guardian analysis → see OPA decision → see mock gateway allow/block/approval → inspect live timeline → stop or complete run → review score and evidence.

Judge-facing proof: Show one benign task that completes, one adversarial action that is blocked, and one sensitive action that pauses for approval. Explain that the payload is generated at runtime and the action is executed only in the isolated mock environment.

Evidence: Show event timestamps, action IDs, policy IDs, tool result, state transition and metric denominator. Do not present pre-recorded events as live execution.

26. Deployment and Environment

Frontend: Deploy React frontend to a suitable static/frontend host. Vercel is an option if compatible with the chosen frontend setup.

Backend: Deploy FastAPI and its background execution process to a Python-capable host. Confirm WebSocket and worker support before selecting the host.

Database: Use Supabase-managed PostgreSQL, Auth and optionally Realtime. Configure RLS and backend service credentials carefully.

OPA: Run OPA as a private service or sidecar reachable only by the backend. Do not expose policy administration publicly.

Presidio: Run Presidio service privately or as a backend-managed component. Confirm resource requirements.

Environment variables: GEMINI_API_KEY, GEMINI_MODEL, SUPABASE_URL, SUPABASE_ANON_KEY, SUPABASE_SERVICE_ROLE_KEY, DATABASE_URL, OPA_URL and service-specific secrets. Values belong in secret management, not source control.

Deployment caveat: The final hosting combination must be verified for persistent workers, WebSockets, timeouts and network access. Do not assume a frontend hosting platform alone can host the complete Python orchestration service.

27. Technical Decisions and Open Questions

Model selection: Choose a currently available Gemini model after a small latency, cost, structured-output and rate-limit test. Keep the model configurable.

LangGraph persistence: Decide whether to use LangGraph's checkpoint integration with PostgreSQL or a minimal custom checkpoint store. Avoid duplicating all state in multiple stores.

Event source of truth: Recommended: persist events in PostgreSQL and publish them over FastAPI WebSockets; optionally use Supabase Realtime for observers. Select one authoritative event sequence.

Background execution: Choose a worker/queue approach compatible with the 24-hour hosting environment. Ensure it supports bounded steps and resumable approval.

OPA deployment: Decide local container versus hosted private service and test backend connectivity.

Presidio scope: Start with a limited set of PII recognizers relevant to synthetic test data; expand only after measuring overhead.

Frontend framework: Use React + Tailwind as required. Select Vite or Next.js frontend based on team familiarity; FastAPI remains the sole application backend.

Unresolved: Confirm hosting provider, concurrency target, simulation step limit, event retention period, approval expiry policy and exact Gemini model before implementation.

28. Future Scope

Extensions: Additional LLM providers; richer scenario packs; configurable policies; multi-agent competition; scenario authoring; experiment comparisons; evaluator API; advanced reports; replayable simulations; enterprise identity integration; additional synthetic tools.

Boundary: Any future integration with real enterprise systems requires a separate threat model, explicit authorization, least-privilege design, security review and revised product scope.

29. Conclusion

Summary: CYBER HUNT is designed as a real-time, observable and sandboxed AI-agent security arena. Gemini powers the agents, LangGraph coordinates their workflow, Presidio helps identify sensitive data, OPA independently evaluates authorization, and the mock gateway enforces the decision. FastAPI provides the backend and Supabase PostgreSQL stores the durable simulation record.

Final principle: AI agents may reason and recommend, but they never receive direct authority to execute sensitive actions. Enforcement remains independent, deterministic, auditable and isolated.